Técnicas de Programación Segura

Unidad de Trabajo 5

Contenidos

- Aplicar principios de programación segura en el desarrollo de servicios.
- Proteger datos sensibles en reposo y en tránsito.
- Implementar mecanismos de autenticación, autorización y control de acceso.
- Usar criptografía para proteger información y credenciales.

Principios/prácticas programación segura

- **Principios básicos**:
  - Mínimo privilegio
  - Defensa en profundidad
  - Fallar de forma segura
  - No confiar en la entrada del usuario

Principios/prácticas programación segura

- **Vulnerabilidades comunes** en servicios:
  - Inyección de código
  - Buffer overflow
  - Gestión insegura de credenciales
  - Escalada de privilegios
  - Condiciones de carrera
  - Exposición de información sensible

Principios/prácticas programación segura

- Encuentra la vulnerabilidad…

```python
import sqlite3

def login(username, password):
    conn = sqlite3.connect('usuarios.db')

    # ! VULNERABLE: concatenación directa de strings
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}';"

    cursor = conn.execute(query)
    result = cursor.fetchone()
    if result:
        return "Login exitoso"
    return "Credenciales inválidas"
```

¡Casos reales!

Principios/prácticas programación segura

- Encuentra la vulnerabilidad…

```c
#include <stdio.h>

void mostrar_entrada(char *input) {
    printf(input);  // ⚠ ! Vulnerable a format string
}

int main() {
    char buffer[100];
    scanf("%99s", buffer);
    mostrar_entrada(buffer);
    return 0;
}
```

¡Casos reales!

Principios/prácticas programación segura

- Encuentra la vulnerabilidad…

```
app.post('/register', function(req, res) {
    const { username, password } = req.body;
    db.save({ username, password }); // texto plano
    res.send('Usuario guardado');
});
```

¡Casos reales!

Principios/prácticas programación segura

- Encuentra la vulnerabilidad…

```php
$cmd = $_GET['cmd'];
$output = shell_exec($cmd);
echo "<pre>$output</pre>";
```

¡Casos reales!

Principios/prácticas programación segura

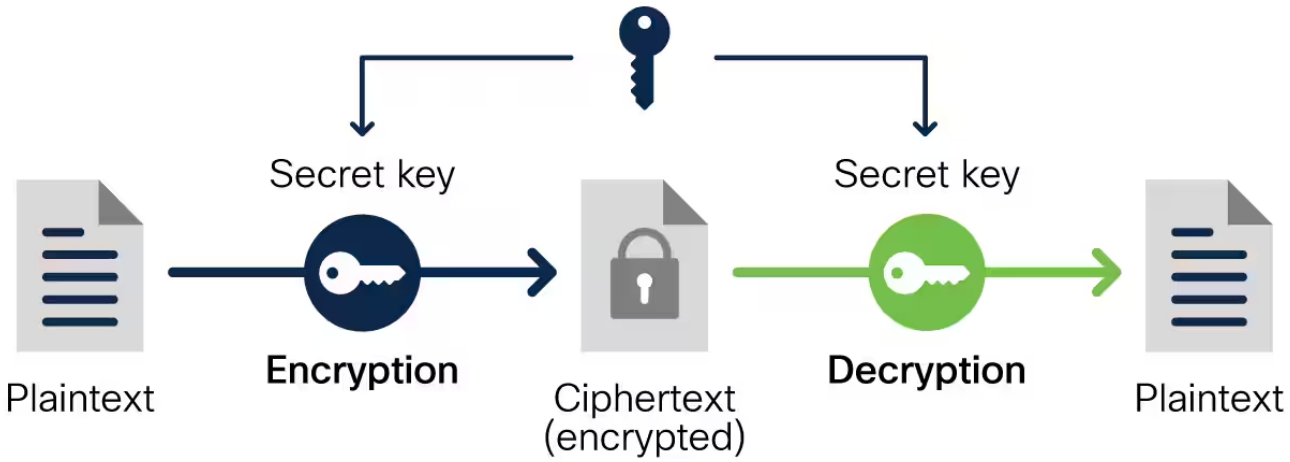
- **Buenas prácticas**:
  - Validación y saneado de entradas
  - Uso de permisos/roles y autenticación
  - Cifrado de información, en reposo o en tránsito
  - Manejo seguro de concurrencia, errores y excepciones
  - Registro (logging) sin exponer datos sensibles

Técnicas y prácticas criptográficas

- **Criptografía aplicada al desarrollo de servicios:**
  - Criptografía simétrica (AES)
  - Criptografía asimétrica (RSA)
  - Funciones hash (SHA-256)

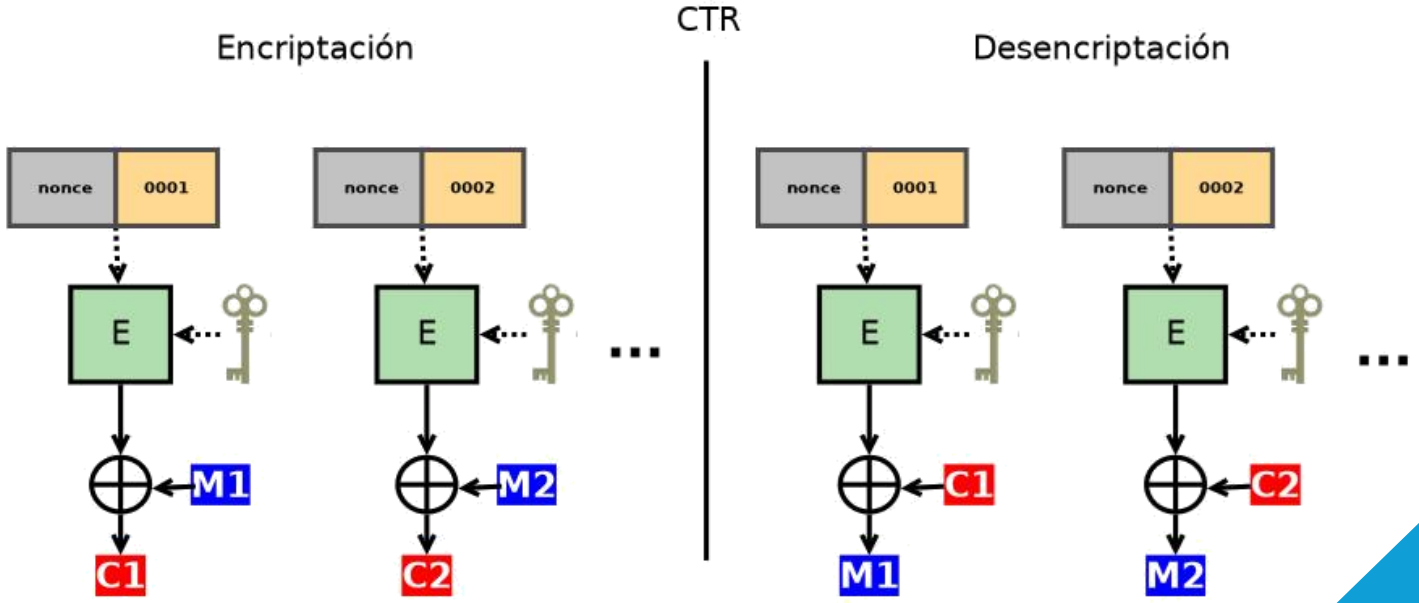
Técnicas y prácticas criptográficas

- **Criptografía SIMÉTRICA:**
  - Más usado: Advanced Encryption System (**AES**)
  - Clave: ≥**128bits**



Técnicas y prácticas criptográficas

- **Criptografía SIMÉTRICA:**
  - Tipo de cifra más usado: Galois/Counter Mode (**CGM**)
  - Contador + Vector de inicialización (IV) ≈ nonce



Técnicas y prácticas criptográficas

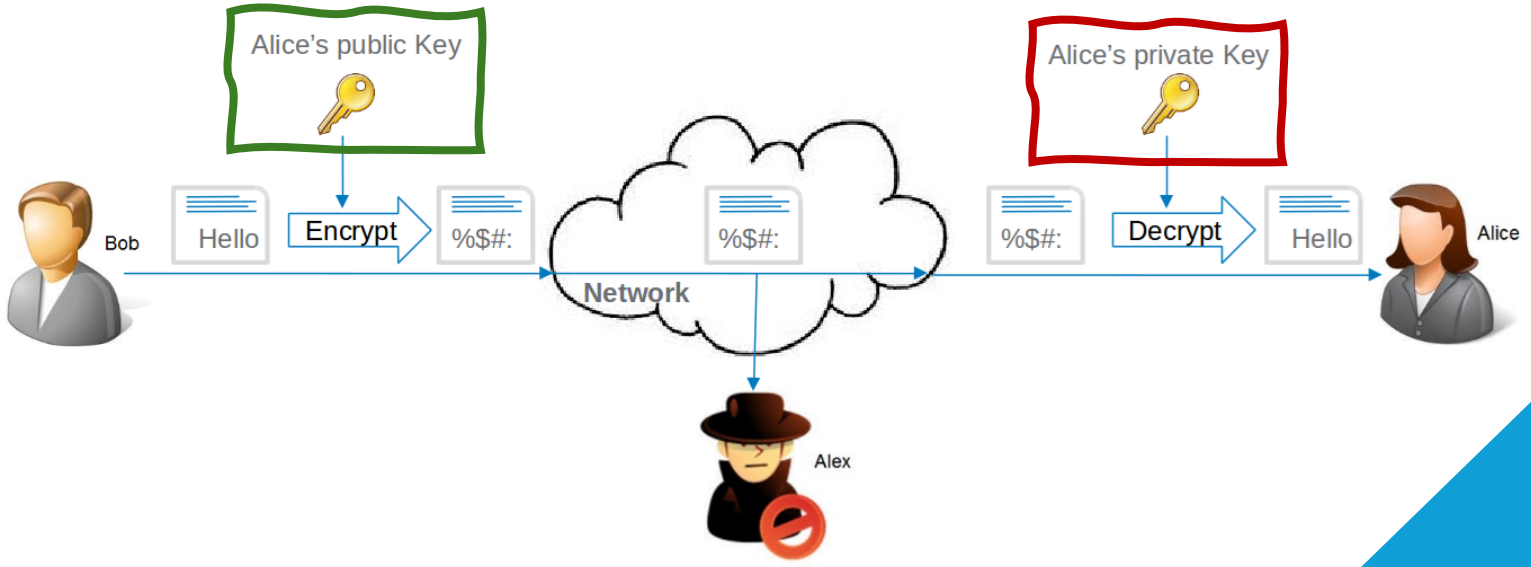
- **Criptografía SIMÉTRICA:**
  - Tipo de cifra más usado: Galois/Counter Mode (**CGM**)
  - Clave + Vector de inicialización (IV ≈ nonce)

```
SecretKey key = KeyGenerator.getInstance("AES").generateKey();

Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
GCMParameterSpec spec = new GCMParameterSpec(128, iv); // 128-bit auth tag
cipher.init(Cipher.ENCRYPT_MODE, key, spec);

byte[] cipherText = cipher.doFinal(plainText.getBytes());
```

  - Se debe almacenar el IV junto a los datos cifrados para usarlo en el descifrado.

Técnicas y prácticas criptográficas

- **Criptografía ASIMÉTRICA:**
  - Más usado: Rivest, Shamir y Adleman (**RSA**)
  - Clave: ≥**2048bits**



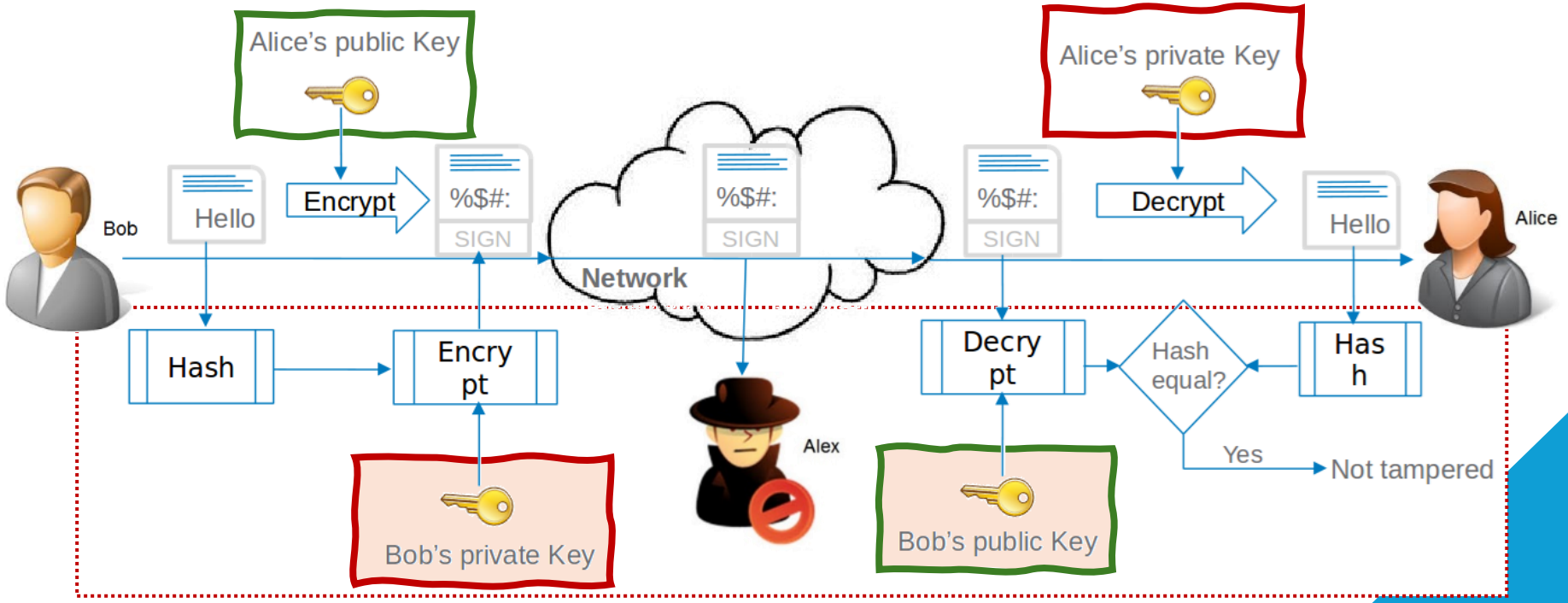
Técnicas y prácticas criptográficas

- **Cifrado vs firma digital**

Uso	Objetivo	Clave usada al iniciar	Resultado
Cifrado	Garantizar la **confidencialidad** del mensaje.	Clave **pública** del receptor.	Solo el receptor puede leerlo con su **clave privada**.
Firma Digital	Garantizar la **autenticidad e integridad** del mensaje.	Clave **privada** del emisor.	Cualquiera puede verificar la firma con la **clave pública** del emisor.

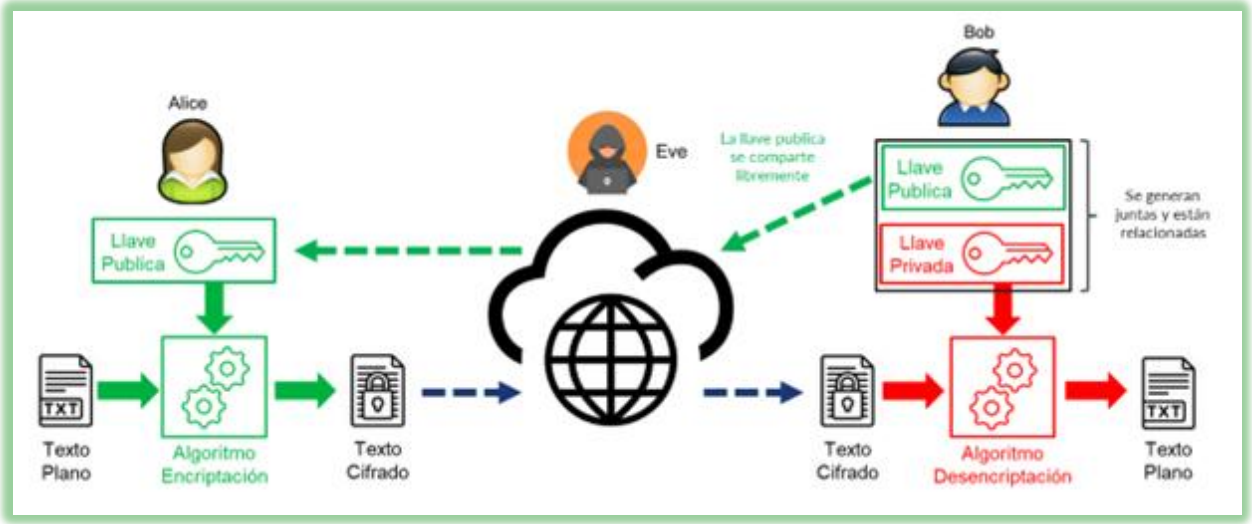
Técnicas y prácticas criptográficas

- **Firma digital**



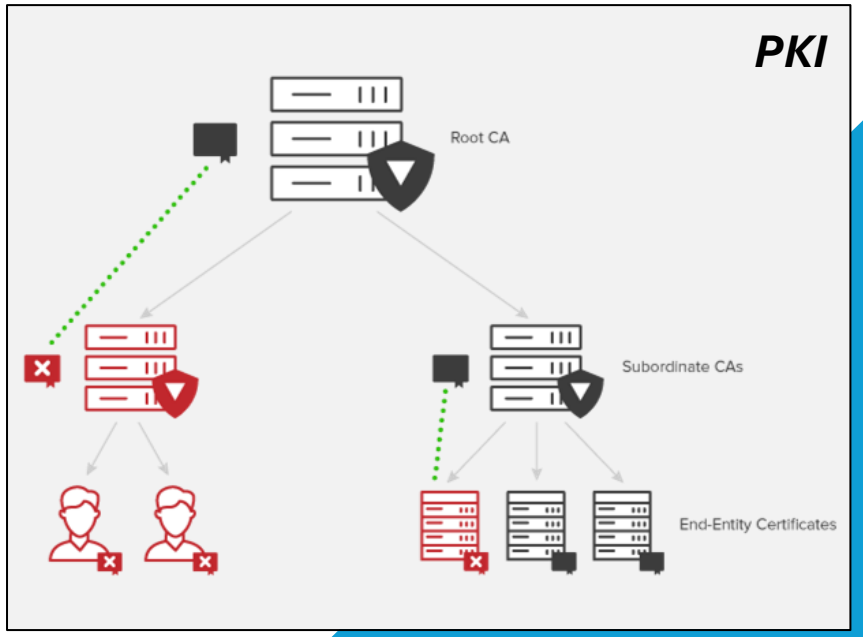
Técnicas y prácticas criptográficas

- **Intercambio seguro de claves de cifra**



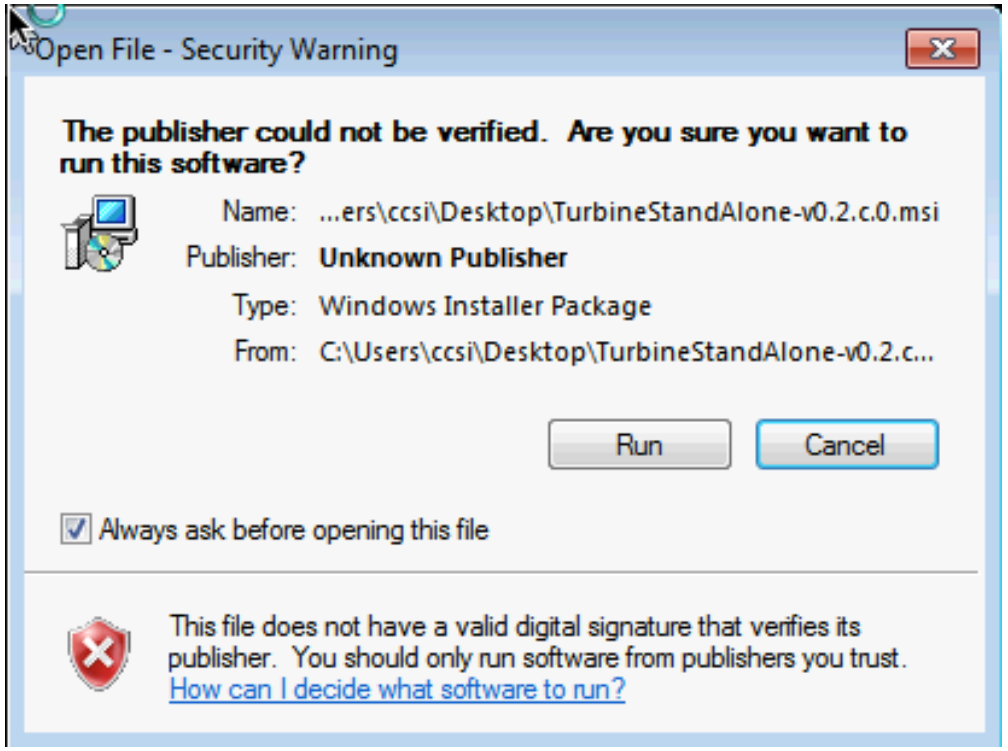
Técnicas y prácticas criptográficas

- **Uso de certificados digitales**
  - HTTPS / TLS (certificados de servidores web).
  - Autenticación usuarios/equipos (VPN, Wi-Fi 802.1X).
  - Firma de software y controladores



Técnicas y prácticas criptográficas

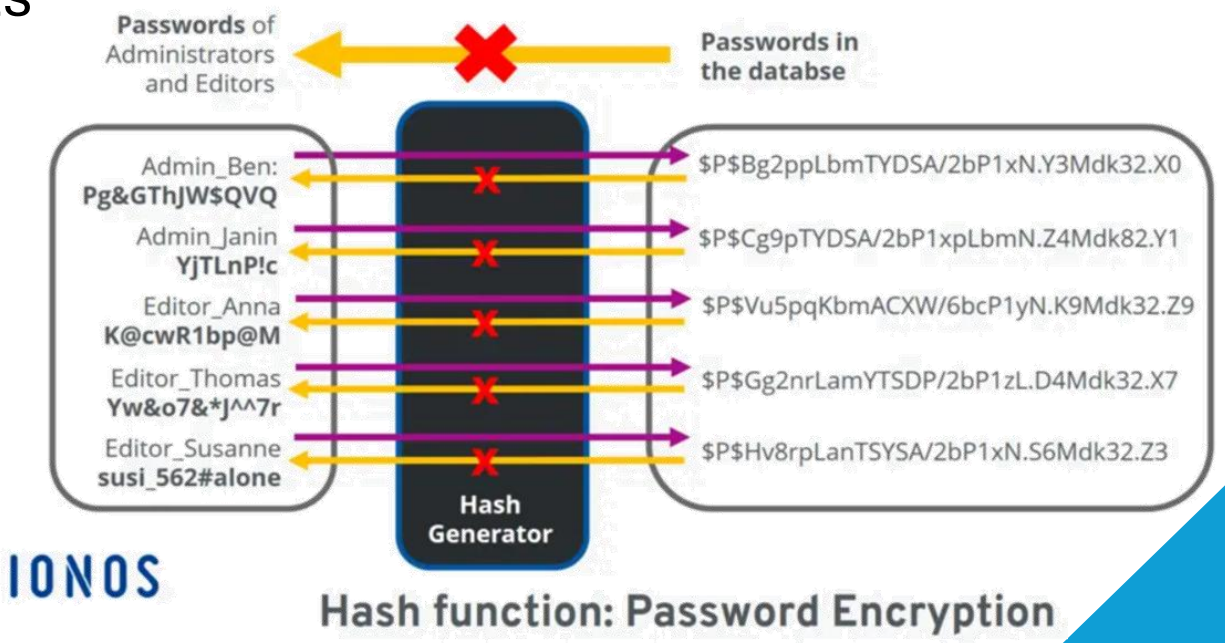
- **Firma de software**



Ciberataque a SolarWinds (cadena de suministro)

Técnicas y prácticas criptográficas

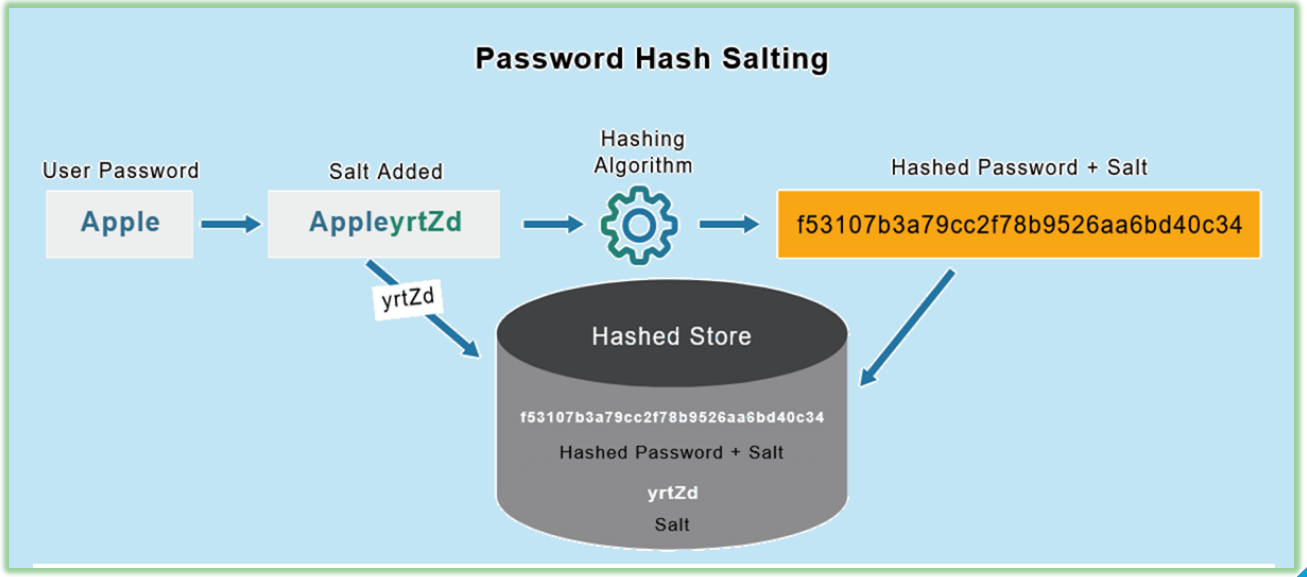
- **Funciones hash (irreversibles):**
  - Más usado: Secure Hash Algorithm (**SHA**)
  - Clave: ≥256bits



Técnicas y prácticas criptográficas

- **Funciones hash:**
  - Uso de “salt”

Password	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz
Salt	-	-	et52ed	ye5sf8
Hash	f4c31aa	f4c31aa	1vn49sa	z32i6t0



Técnicas y prácticas criptográficas

- **Generando hashes:**
  - PBKDF2 es una función de derivación de claves, (no un hash “simple”) que podemos usar en Java.
  - Su misión es:
    - Tomar una contraseña (débil)
    - Aplicar muchas iteraciones
    - Usar una función hash interna
    - Producir una clave criptográfica de la longitud que pidamos (p.e.: 256bits)

Técnicas y prácticas criptográficas

- **Generando hash de una contraseña con PBKDF2:**
  - Contraseña + salt

```
byte[] salt = new byte[16];
new SecureRandom().nextBytes(salt);

PBEKeySpec spec = new PBEKeySpec(password, salt, iterations, keyLength);
SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");

byte[] hash = skf.generateSecret(spec).getEncoded();
```

  - Se almacena el salt junto a los datos cifrados para usarlo en posteriores comprobaciones.

Seguridad en almacenamiento de información

- Protección de datos en reposo
  - **Cifrado** de ficheros y bases de datos
  - Almacenamiento con **hash de contraseñas**
- Gestión de claves
  - No incrustar claves en el código
  - Uso de variables de entorno
- Control de acceso a recursos locales
  - Acceso por roles
  - Esquemas RBAC (Role-Based Access Control)

Se refiere a **proteger la información cuando está almacenada** y no se está transmitiendo:

- Bases de datos
- Ficheros
- Logs
- Configuraciones
- Copias de seguridad

Seguridad en almacenamiento de información

- *“Por el amor del cielo”*, no almacenéis información sensible:
  - En ficheros sin configurar los adecuados privilegios.
  - Sin cifrar adecuadamente (codificar con base64, o similares, NO ES CIFRAR).
- Si la información no requiere ser revertida (i.e. passwords), siempre se puede hacer un hash (SHA-256 o similar/superior).

Seguridad en almacenamiento de información

- Gestión de permisos
  - Java tiene **dos APIs distintas**:

API	Desde	Uso
java.io.File	Java 1.0	**Muy limitada**
java.nio.file (Files)	Java 7	**Correcta y completa**

    - File solo sirve para casos muy básicos o compatibilidad:
      i. Puede fallar silenciosamente
      ii. No controla quién lee
      iii. No revoca permisos ACL
    - **Prioridad de uso → Files** (NIO.2)

Seguridad en almacenamiento de información

- Gestión de permisos (modificación)
  - Java.NIO en sistemas de ficheros POSIX (Linux, Unix, macOS)

```
// Localizamos el fichero en la ruta relativa del proyecto
Path path = Path.of("secretos.txt");

if(UtilidadesSistema.isLinux()) {
   // Leemos los permisos
   Set<PosixFilePermission> perms=
         Files.getPosixFilePermissions(path);
   System.out.println(perms);

   // Asignamos permisos. Equivale a un chmod 600 secretos.txt
   perms= PosixFilePermissions.fromString("rw-------");
   Files.setPosixFilePermissions(path, perms);
```

Seguridad en almacenamiento de información

- Gestión de permisos (creación)
  - Java.NIO en sistemas de ficheros POSIX (Linux, Unix, macOS)

```
if(UtilidadesSistema.isLinux()) {
    // Establecemos el set de permisos a aplicar al nuevo fichero
    Set<PosixFilePermission> perms =
            PosixFilePermissions.fromString("rw-------");

    // Definimos un atributo con los permisos deseados
    FileAttribute<Set<PosixFilePermission>> attr =
            PosixFilePermissions.asFileAttribute(perms);

    // Creamos directamente el fichero aplicándole el atributo
    Files.createFile(path, attr);
```

Seguridad en almacenamiento de información

- Gestión de permisos
  - Errores habituales en sistemas de ficheros POSIX (Linux, Unix, macOS):
    - Generar directorios sin permiso x (no se podrían atravesar)
    - Crear fichero y luego cambiar permisos
    - Usar permisos 777
    - Confiar en umask
    - Ejecutar como root sin control

En seguridad el fichero debe **nacer protegido**

Seguridad en almacenamiento de información

- Gestión de permisos (modificación)
  - Java.NIO en sistemas Windows basada en ACLs

```
if (UtilidadesSistema.isWindows()) {

    // Obtenemos vista de la ACL (Access Control List) del fichero
    AclFileAttributeView aclView =
            Files.getFileAttributeView(path, AclFileAttributeView.class);

    // Obtenemos identidad del usuario al que afectan los cambios
    UserPrincipal user =
            path.getFileSystem()
                    .getUserPrincipalLookupService()
                    .lookupPrincipalByName(userName);
```

```
// Configuramos nueva entrada para la ACL del usuario
AclEntry entry = AclEntry.newBuilder()
        .setType(AclEntryType.ALLOW)
        .setPrincipal(user)
        .setPermissions(AclEntryPermission.READ_DATA)
        .build();
List<AclEntry> acls = new ArrayList<>();
acls.add(entry);

// Modificamos la vista sobre el fichero con las nueva ACL
aclView.setAcl(acls);
```

Seguridad en almacenamiento de información

- Gestión de permisos (creación)
  - Java.NIO en sistemas Windows basada en ACLs

    En lugar del *aclView.setAcl(acls)*

```
// Atributo ACL
FileAttribute<List<AclEntry>> attr =
    new FileAttribute<>() {
        @Override
        public String name() {
            return "acl:acl";
        }

        @Override
        public List<AclEntry> value() {
            return acl;
        }
    };

// Crear fichero con ACL inicial
Files.createFile(path, attr);
```

Seguridad en almacenamiento de información

- Gestión de permisos
  - Errores habituales en sistemas Windows:
    - Crear fichero y luego cambiar ACL
    - No asignar suficientes permisos
    - Confiar en setReadOnly()
    - Buscar usuarios por nombre incorrecto
    - Olvidar herencia

      SYSTEM y BUILTIN\Administrators suelen tener acceso, aunque no se configure si hay herencia

Seguridad en almacenamiento de información

- Gestión de permisos
  - En POSIX/Linux también se pueden configurar ACLs para establecer permisos con más precisión.
  - Java **NO tiene una API** para ello como en el caso de Windows.
  - Deben hacerse llamadas al sistema con **ProcessBuilder**.

```
// Creamos directamente el fichero aplicándole el atributo
Files.createFile(path, attr);

ProcessBuilder pb = new ProcessBuilder(
        "setfacl",
        "-m",
        "u:david:r",
        "/datos/secretos.txt"
);

Process p = pb.start();
int exit = p.waitFor();

if (exit != 0) {
    throw new RuntimeException("Error aplicando ACL");
}
```

Políticas de seguridad y control de acceso

- **Autenticación vs autorización**
- **Políticas** de seguridad:
  - Acceso por usuario
  - Acceso por roles
- **Esquemas RBAC** (Role-Based Access Control)
- Ejemplos:
  - Rol administrador / usuario
  - Restricción de operaciones según rol

Seguridad en la transmisión de información

- **Riesgos en la transmisión de datos:**
  - Interceptación
  - Man-in-the-middle
- **Métodos para asegurar la transmisión:**
  - TLS/SSL
  - Cifrado de datos antes de enviar
- **Uso de:**
  - HTTPS
  - Sockets seguros (SSL sockets))